

Guía de Configuración: Entorno Profesional Airflow + PySpark + MinIO

Autor: Lauro IT

Este documento detalla la configuración profesional realizada para migrar un pipeline de datos local a un entorno de contenedores Docker, garantizando persistencia, escalabilidad y automatización.

1. Arquitectura del Entorno

Se ha implementado una arquitectura de contenedores robusta que incluye:

- **Orquestador:** Apache Airflow 2.7.0.
- **Procesamiento:** PySpark (Spark 3.5.0) con soporte para Java JRE.
- **Almacenamiento (Data Lake):** MinIO (S3 compatible) con buckets para capas Bronze y Silver.
- **Base de Datos:** PostgreSQL para los metadatos de Airflow.

2. Configuración de Persistencia (Dockerfile)

Para evitar reinstalar Java y dependencias manualmente en cada reinicio, se creó un archivo Dockerfile (sin extensión .txt). Este archivo automatiza la instalación de las herramientas necesarias dentro de la imagen de Airflow.

```
FROM apache/airflow:2.7.0
USER root
RUN apt-get update && apt-get install -y default-jre && apt-get clean
&& rm -rf /var/lib/apt/lists/*
USER airflow
RUN pip install findspark pyspark
```

3. Ajustes en Docker Compose

Se modificó el archivo docker-compose.yaml para que los servicios **Webserver** y **Scheduler** utilicen la imagen personalizada construida por el Dockerfile anterior.

Servicio	Cambio Realizado
airflow-webserver	Se comentó image: y se agregó build: .
airflow-scheduler	Se comentó image: y se agregó build: .
Volumes	Mapeo de la carpeta ./scripts a /opt/airflow/scripts para acceso a archivos .py, CSV y JSON.

4. Optimización de Scripts de Spark

Los scripts fueron limpiados de rutas locales de Windows para ser compatibles con

Linux/Docker:

- **Eliminación de Rutas C:\:** Se removieron configuraciones de HADOOP_HOME y extraClassPath que apuntaban al disco local.
- **Rutas Relativas al Contenedor:** Los archivos se leen ahora desde /opt/airflow/scripts/.
- **Conectividad S3A:** Configuración optimizada para MinIO utilizando los paquetes hadoop-aws y aws-java-sdk-bundle.

5. Comandos de Mantenimiento

Para aplicar cambios estructurales en el Dockerfile o el Compose:

```
docker-compose down  
docker-compose up -d --build
```

Para monitorear el estado de los recursos y descargas de Spark:

```
docker stats airflow-scheduler  
docker logs -f airflow-scheduler
```

6. Mejores Prácticas Aplicadas

1. **Uso de Parquet:** Se utiliza el formato Parquet para las capas Bronze y Silver por su eficiencia en almacenamiento y consulta.
2. **Estrategia de Carga:** Implementación de overwrite para asegurar que las pruebas no dupliquen datos.
3. **Manejo de Errores:** Inclusión de bloques try/except/finally con spark.stop() para liberar recursos.

Fin del instructivo técnico.